GATHR · BEFORE MILESTONE 9

Shipping Gathr.

You've built a full-stack app on your laptop. This guide explains what each part of it actually is, where each part runs, and what changes when it moves from your Mac to the internet. Every example is a file you already wrote.

01 The parts of a web app

02 Frontend vs backend in Next.js

03 What a "server" really is

04 Docker and your local database

05 Gathr on your Mac today

06 Build vs run

07 Gathr on the internet

08 What has to change

09 Secrets and env variables

10 The database in production

11 From git push to live site

12 After it's live

13 Our M9 checklist

14 Glossary

01

The parts of a web app

Every web app, from Instagram to Gathr, is the same few parts talking to each other. When you've only written React, you've mostly lived in the first one.

frontend

Code in the browser

Buttons, forms, the map, the chat box. Runs on the visitor's phone or laptop. Anyone can open DevTools and read it, so it can't hold secrets.

backend

Code on a server

Checks who you are, decides if you're allowed, talks to the database. Runs on a computer you control. Visitors only see what it sends back.

database

Where data lives

Users, plans, join requests, messages. A separate program (Postgres) that the backend talks to. The browser never talks to it directly.

services

Helpers

Things your app uses but doesn't run itself. For Gathr: photo storage and sending sign-in emails.

ANALOGY

A restaurant. The frontend is the dining room and menu. The backend is the kitchen, where staff check orders and cook. The database is the pantry. Customers never walk into the pantry; they ask a waiter, and the kitchen fetches what's allowed.

02

Frontend vs backend in Next.js

In a classic React setup these are two separate projects: a Vite app (frontend) and, say, an Express API (backend), deployed to two different places. **Next.js puts both in one project.** The same `src/` folder contains code that runs on the server and code that runs in the browser, and the build splits them apart.

The rule that decides where a file runs:

- **No directive (the default):** a *Server Component*. Runs on the server only. It can read the database and secrets. The browser gets the HTML it produced, never the code.
- `"use client"` **at the top:** a *Client Component*. Rendered to HTML on the server first, then its JavaScript is sent to the browser so it can respond to clicks (this is "hydration").
- `"use server"`**:** a *Server Action*. A function the browser can call, but it runs on the server. Next.js turns the call into an HTTP request for you.

GATHR FILE	RUNS ON	WHY
`app/(app)/explore/page.tsx`	server	Server Component: reads the URL filters and queries the DB.
`features/discover/queries.ts`	server	The PostGIS "plans near me" SQL. Needs the database password.
`features/join/actions.ts`	server	Server Action behind "I'm in". Checks auth, then runs the locked transaction so a plan can't overbook.
`src/proxy.ts`	server	Runs before every request and redirects signed-out users away from private pages.
`api/activities/[id]/messages/route.ts`	server	A Route Handler, the Next.js version of an Express endpoint. The chat polls it.
`features/chat/components/chat-room.tsx`	server → browser	`"use client"`: first HTML from the server, then lives in the browser and polls every 4 s.
`features/discover/components/explore-map.tsx`	browser	Leaflet needs `window`, so it's loaded with `ssr: false` and runs only in the browser.
`lib/auth.ts` vs `lib/auth-client.ts`	server / browser	Same library, two halves. The server half holds `BETTER_AUTH_SECRET`; the client half only calls it.

So "deploying the frontend and the backend" for Gathr is *one* deploy. The host builds the project once, then serves the browser files and runs the server code.

03

What a "server" really is

A server is just **a program that stays running and waits for requests**, on a computer that stays switched on. When you run `npm run dev`, your Mac *is* the server. It listens on a **port**, which is like a numbered door on the computer.

ADDRESS	WHAT'S LISTENING
`localhost:3000`	Next.js dev server (`npm run dev`), your app
`localhost:3100`	The production build we started for Lighthouse (`next start`)
`localhost:5432`	Postgres, inside Docker

`localhost` means "this same computer". That's why your friend can't open your `localhost:3000`: on their phone, localhost is *their* phone. Deploying means running the same server on a computer that has a public address (like `gathr.vercel.app`), is always on, and that anyone can reach.

04

Docker and your local database

Postgres is a program, like Chrome or Spotify. You could install it directly on macOS, but then you'd also need PostGIS, matching versions, and cleanup later. **Docker runs it in a container instead**: a sealed, pre-packaged mini Linux computer with Postgres + PostGIS already installed, running inside your Mac.

> ANALOGY
> An *image* is a recipe card (`imresamu/postgis:17-3.5`). A *container* is one dish cooked from it (`gathr-db`). A *volume* is the fridge the leftovers go in (`gathr_pgdata`): throw the dish away and cook a new one, and the data is still in the fridge.

Your `docker-compose.yml` is a written order for that dish:

```
# docker-compose.yml (simplified)
services:
  db:
    image: imresamu/postgis:17-3.5     # which recipe
    container_name: gathr-db           # the container's name
    environment:
      POSTGRES_USER: gathr             # login it creates on first start
      POSTGRES_PASSWORD: gathr
    ports:
      - "5432:5432"                    # Mac port 5432 → container port 5432
    volumes:
      - gathr_pgdata:/var/lib/postgresql/data   # data survives restarts
```

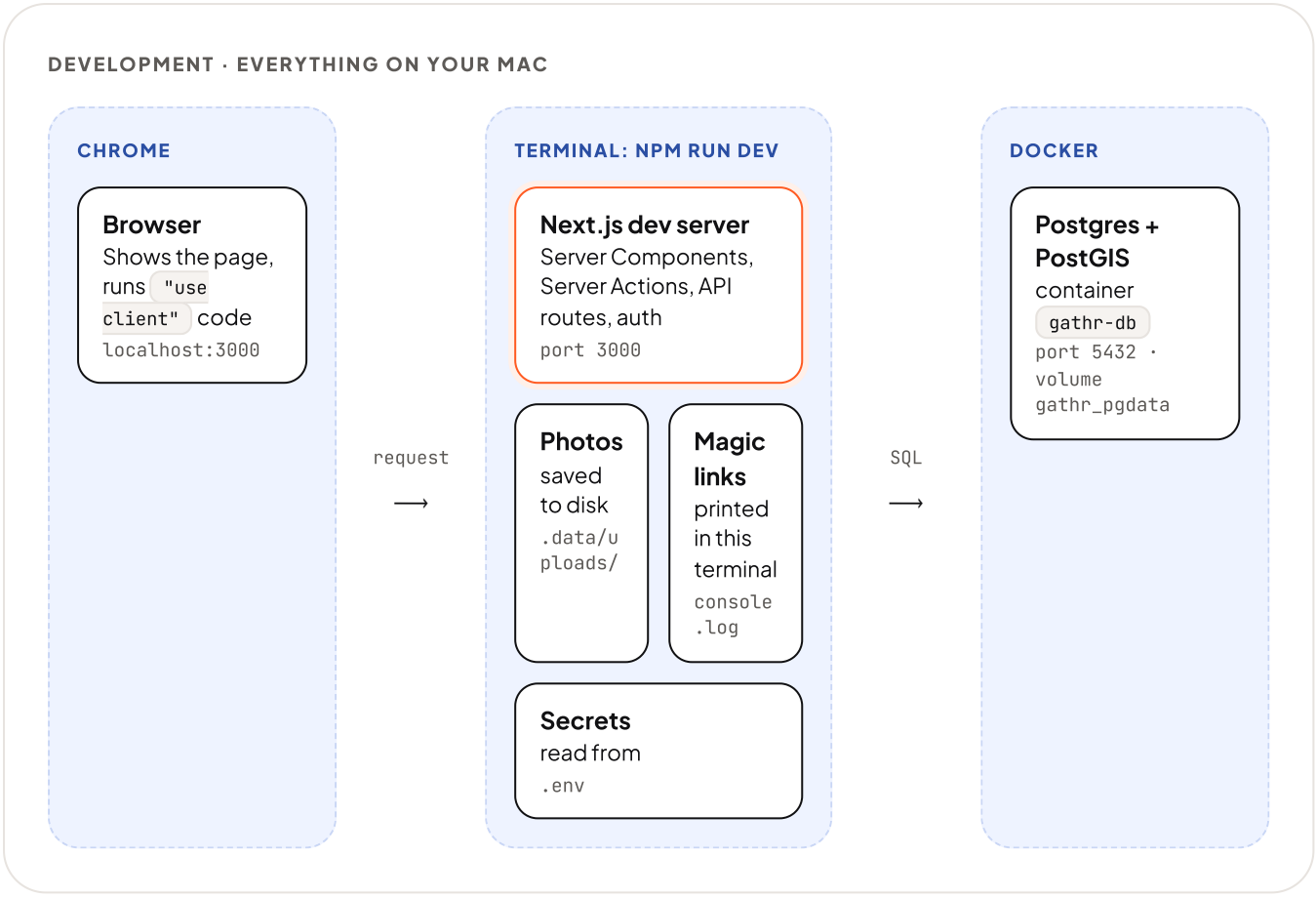
COMMAND	WHAT IT DOES
`docker compose up -d`	Start the database in the background (`-d` = detached)
`docker ps`	List running containers. You should see `gathr-db … (healthy)`
`docker compose down`	Stop it. Data is kept in the volume
`docker compose down -v`	Stop it **and delete the volume**, wiping all local data

Docker is a *local* convenience here. In production we don't ship your container anywhere. We rent a Postgres that someone else runs and backs up, and point the app at it by changing one line: `DATABASE_URL`.

05

Gathr on your Mac today

Everything runs on one machine. Follow the arrows when you tap "I'm in" on a plan:



Three things in this picture only work because it's your own laptop: the photos folder, reading magic links from your terminal, and a database that lives in Docker. Keep them in mind; they're exactly what M9 changes.

06

Build vs run

A deploy has two phases, and you've already done both locally:

`npm run build`

Build: once per deploy

Type-checks, compiles TypeScript, bundles and minifies the browser JavaScript, optimises CSS, and prerenders every page it can into HTML. Output goes in `.next/`.

`npm start`

Run: for every visitor

Serves the built files and runs the server code for each request. Fast because the build did the heavy lifting.

`npm run dev`

Dev: neither

Compiles each page on demand as you visit it and reloads on save. Great for coding, slow and unoptimised. Never what users get.

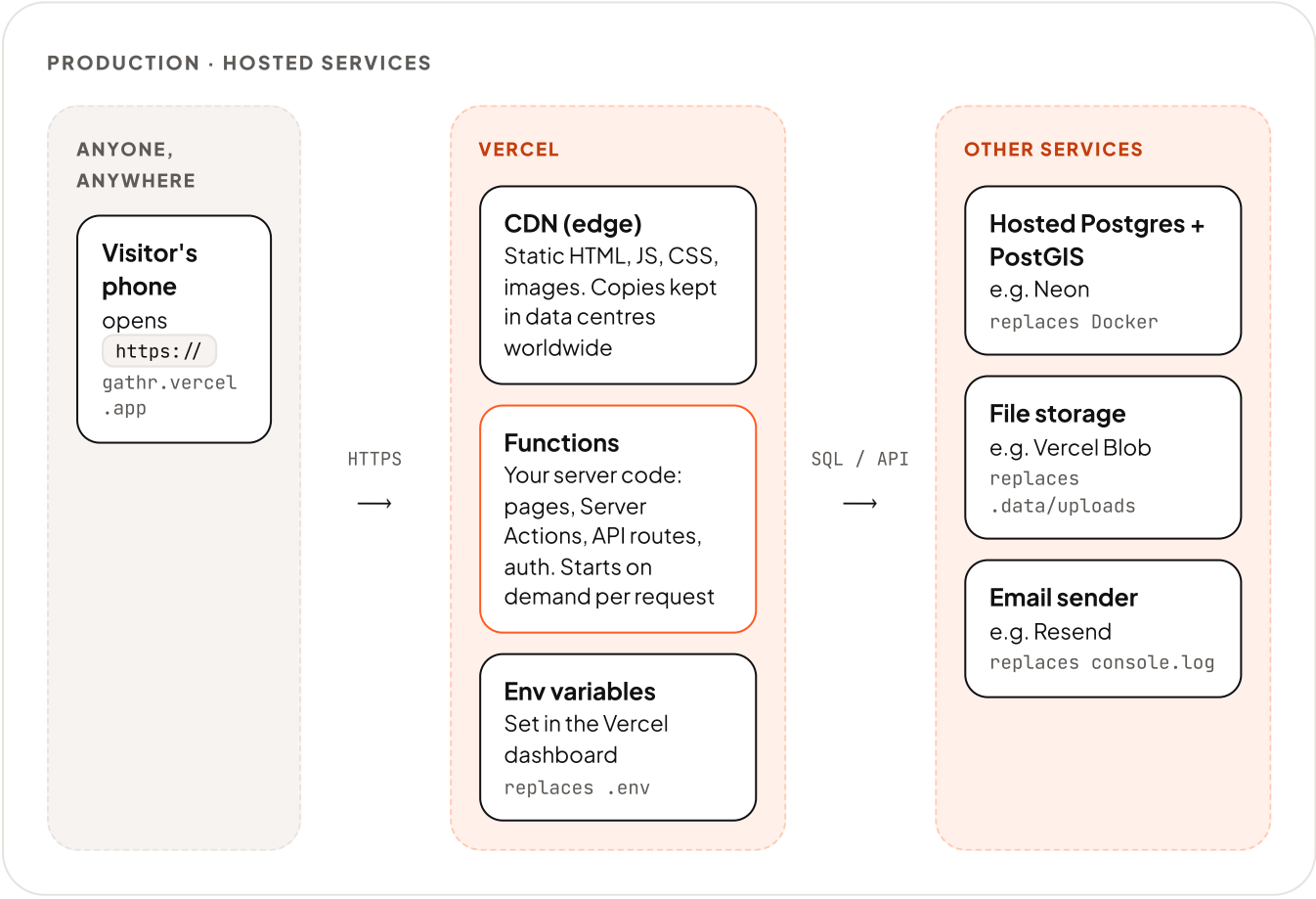
Remember the symbols at the end of `npm run build`? They tell the host how to serve each route:

SYMBOL	MEANING	GATHR EXAMPLE
○ Static	Fully made at build time. Served instantly from a CDN, no server work.	The landing page `/`
◐ Partial prerender	The shell (header, layout) is prerendered; the personal parts stream in from the server.	`/plans`, `/sign-in`
ƒ Dynamic	Made fresh on the server for each request.	`/uploads/[...path]`

07

Gathr on the internet

Same app, but each laptop-only part is replaced by a service that's always on. This is the setup BRD §11 plans for M9:



Why Vercel has no "photos folder"

Locally, one server runs continuously and has a hard drive. On Vercel your server code runs as **functions**: small copies of your server that start up when requests arrive, possibly several at once in different places, and shut

down when idle. Each copy gets a blank, temporary disk. A photo saved by one copy is invisible to the next and gone when it shuts down. So anything that must last goes to a service built for it: the database for data, a storage bucket for files.

> ANALOGY
> Your laptop is a house with a cupboard. Vercel is a hotel: you get a fresh room for each stay, and housekeeping clears it when you leave. Keep your valuables in a bank (the database and storage), not in the room.

08

What has to change

Good news: most code doesn't change at all. Here's the full list for Gathr:

PART	ON YOUR MAC	IN PRODUCTION	CODE CHANGE
Database	Docker `gathr-db`	Hosted Postgres with PostGIS	None. Just a different `DATABASE_URL`
Photos	`.data/uploads/`	Storage bucket	Rewrite `lib/storage.ts` (3 functions; the rest of the app already goes through them)
Sign-in email	Printed in terminal	Real email via a sending service	Fill in `sendMagicLink` in `lib/auth.ts`
Secrets	`.env` file	Vercel dashboard	None
App URL	`http://localhost:3000`	`https://…vercel.app`	None. Set `BETTER_AUTH_URL` and `SITE_URL`
Google sign-in	localhost redirect	Add the live URL in Google Cloud	None
DB schema	`prisma migrate dev`	`prisma migrate deploy` on each release	A build step

09

Secrets and env variables

Some values differ between your laptop and production (which database?), and some must never be public (the auth secret, DB password). Both live in **environment variables**, settings the server reads at start-up via `process.env.NAME`.

```
# Locally: .env (git-ignored, never committed)
DATABASE_URL="postgresql://gathr:gathr@localhost:5432/gathr"
BETTER_AUTH_SECRET="…random…"
BETTER_AUTH_URL="http://localhost:3000"

# In production: the same names, typed into Vercel → Settings → Environment Variables
DATABASE_URL="postgresql://user:pass@ep-xyz.neon.tech/gathr?sslmode=require"
BETTER_AUTH_SECRET="…a NEW random value…"
BETTER_AUTH_URL="https://gathr.vercel.app"
```

- `.env` is in `.gitignore`, so it never reaches GitHub. `.env.example` is committed as a template with the names but no values.
- Anything starting with `NEXT_PUBLIC_` is copied into browser JavaScript, where everyone can read it. Never put a secret behind that prefix.
- Production gets its **own** secrets. Never reuse your laptop's.

10

The database in production

The code is easy to replace: deploy again and the old version is gone. **Data is not.** That's why the database gets the most care.

Migrations: git for your table structure

Each folder in `prisma/migrations/` is one recorded change to the schema (`init_postgis`, `data_layer`, `moderation_hidden` …). Postgres keeps a table of which ones it has already applied. A deploy applies only the new ones, in order, without touching existing rows.

COMMAND	USE IT	WHAT IT DOES
`prisma migrate dev`	Laptop only	Compares your schema to the DB, *writes* a new migration, applies it. May offer to reset (wipe) the DB.
`prisma migrate deploy`	Production	Applies already-written migrations. Never generates or resets anything.
`prisma db seed`	Once	Inserts the starting data (categories). Demo plans are optional in production.

Two production-only concerns

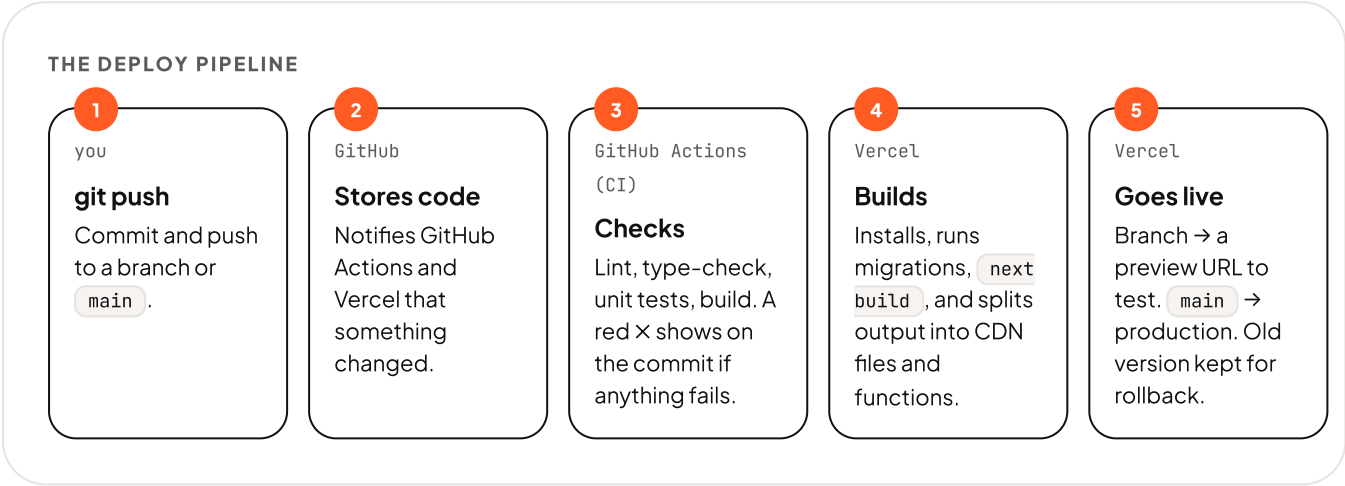
- **Connection pooling.** Many functions starting at once could each open database connections and exhaust the limit. Hosted Postgres providers offer a "pooled" connection string that shares a few connections between all of them; we'll use that.

- **Backups.** Hosted providers take them automatically. That's one big reason not to run your own database server.

11

From git push to live site

Once it's set up, you never "upload" anything by hand. Pushing to GitHub *is* deploying:



CI

Continuous Integration

A robot runs your checks on every push, so broken code is caught before users see it. Ours will be a workflow file in `.github/workflows/`.

CD

Continuous Deployment

Every push to `main` that passes is deployed automatically. That's Vercel's GitHub integration.

preview

Preview deployments

Each branch or PR gets its own temporary URL to try a change in the real cloud setup before merging.

rollback

Instant rollback

A bad release? Promote the previous deployment in Vercel with one click. Old builds are kept.

12

After it's live

Coding is done; running an app has its own routine. What changes for you:

Logs

Your `console.log` output now appears in Vercel → Logs, not your terminal. That's where you look when something breaks.

Errors

Users see the error page you built in M8; the real error and its `digest` ID are in the logs. An error tracker (e.g. Sentry) can alert you, later.

Schema changes

Change `schema.prisma` → `migrate dev` locally → commit the migration → push. The deploy applies it. Avoid deleting or renaming columns that hold real data.

Admin

Run `make-admin` once against the production DB for your account, then moderate from `/admin`.

Domain

Optional: buy a domain (roughly ₹800–1,500 a year for a .com) and point it at Vercel. HTTPS certificates are automatic.

Costs

Free tiers cover a learning project: Vercel Hobby, Neon free, Blob and Resend free allowances. Free databases may sleep when idle, so the first request after a quiet spell is a little slower.

Our M9 checklist

What we'll do together, in order. Steps marked you need your own accounts; I'll walk you through each screen.

- [ ] **Pick services:** database, photo storage, email (and whether you have a domain).
- [ ] **Code:** cloud storage in `storage.ts`, real emails in `auth.ts`, fail-fast checks for missing env variables, migrations in the build.
- [ ] **CI:** a GitHub Actions workflow running lint, types, tests and build on every push.
- [ ] **Create accounts and connect GitHub to Vercel** you
- [ ] **Create the production database**, apply migrations, seed categories. you
- [ ] **Set env variables in Vercel**, with new secrets. you
- [ ] **First deploy**, then `make-admin` for your account.
- [ ] **Smoke test the live URL:** sign in, create a plan with a photo, join from a second account, chat, check the map.
- [ ] **Wrap up:** live URL in the README, notes.md, commit.

Glossary

Frontend	Code that runs in the visitor's browser.
Backend	Code that runs on a server you control. Holds secrets, talks to the database.
Server	A program that waits for requests, on a computer that's always on.
Port	A numbered door on a computer. 3000 = Next.js, 5432 = Postgres.
localhost	"This same computer". Not reachable by anyone else.
Docker image / container / volume	Recipe / running copy / where its data is kept.
Build	Turning source code into optimised files ready to serve (`.next/`).
CDN	Copies of static files stored around the world so they load fast everywhere.
Function (serverless)	Your server code, started on demand per request by the host. No permanent disk.
Env variable	A setting the server reads at start-up. Differs per environment; holds secrets.
Migration	One recorded, ordered change to the database structure.
Connection pooling	Sharing a few database connections among many function copies.
CI / CD	Robots that check every push / deploy every passing push.
Preview deployment	A temporary live URL for a branch, to test before merging.
DNS / domain	The internet's phone book; maps `gathr.app` to the servers that host it.
HTTPS	Encrypted connection (the padlock). Vercel sets it up automatically.

Written for Gathr, before Milestone 9. Service names (Vercel, Neon, Vercel Blob, Resend) follow BRD §11 and are examples until we choose together.